UT2. MANEJO DE CONECTORES

ACTIVIDAD 6

Modificación sobre tabla de base de datos con ResultSet actualizable

Título: Modificación de datos en una Base de Datos MySQL desde Java con ResultSet actualizable.

Descripción:

En esta actividad, implementarás una clase en Java que se conectará a tu base de datos MySQL y usará un ResultSet actualizable para modificar registros en la tabla CLIENTES. La aplicación debe gestionar los errores de conexión y SQL, mostrando el número de registros afectados (puedes mostrarlo con un elemento basado en Java Swing).

Objetivos:

- Conectar a una base de datos MySQL desde una aplicación Java.
- Modificar registros en una base de datos mediante ResultSet con capacidad de actualización.
- Practicar el uso de ResultSet avanzados para operaciones de modificación de datos, gestionando correctamente las excepciones SQL y mostrando el resultado de manera gráfica.

Instrucciones:

1. **Configuración de la Base de Datos:**
   - Utiliza la tabla **clientes** de la base de datos **ut2**, con los 3 clientes que insertaste en la actividad 5 (al menos uno de ellos tiene el CP a NULL). Recuerda que tiene las siguientes columnas:
     - DNI: Identificador único de tipo CHAR(9), no nulo y clave primaria.
     - APELLIDOS: Apellidos del cliente de tipo VARCHAR(32), no nulo.
     - CP: Código postal de tipo CHAR(5), permite valores nulos.
2. **Implementación de la clase de modificación con ResultSet**
   a) **Crear la clase que realiza la modificación**:
      - En tu proyecto Java, crea una clase denominada ModificarClientesResultSet.
      - Define las constantes necesarias (URL, USER, PASSWORD) para gestionar la conexión a la base de datos **ut2**.
   b) **Configurar el ResultSet como actualizable**:
      - Deshabilita el modo auto-commit en la conexión para asegurarte de que los cambios realizados sean parte de una transacción.

- Configura el Statement para que permita desplazamiento bidireccional y modificación de los datos utilizando ResultSet.TYPE_SCROLL_SENSITIVE y ResultSet.CONCUR_UPDATABLE.

c) **Realizar modificaciones en el ResultSet**:
   - Crea un ResultSet con la consulta **SELECT * FROM clientes**. Para que el ResultSet sea actualizable, la consulta debe ser sobre una sola tabla e incluir su clave primaria (DNI).
   - Utiliza métodos de ResultSet como updateString() y updateRow() para modificar el contenido de las filas seleccionadas.
   - Recorre el ResultSet con un bucle while. Para cada fila, comprueba si el código postal es nulo con **getString("CP") == null**.
   - Si es nulo, cámbialo a "00000" con **updateString("CP", "00000")** y guarda el cambio en la fila actual con **updateRow()**.
   - Cuenta en una variable **nFil** cuántas filas has modificado.
   - Al terminar, muestra en la consola el mensaje: "Filas modificadas: [nFil]".
   - Cuando hayas recorrido todas las filas, confirma la transacción con connection.commit(). Sin esta llamada, los cambios se pierden al cerrar la conexión.

3. **Salida gráfica (opcional):**
   - Este apartado no es obligatorio. Si lo realizas, se valorará como funcionalidad extra (nivel 10 de la rúbrica).
   - Además del mensaje en la consola, muestra el **número de filas modificadas** en un **cuadro de diálogo usando Java Swing**.
   - Para ello, utiliza el método **JOptionPane.showMessageDialog(null, mensaje)**, que abre una ventana emergente con el texto indicado. El primer parámetro (null) indica que el cuadro de diálogo no depende de ninguna otra ventana.
   - Los mensajes de error se siguen mostrando siempre en la consola.
   - Si realizas este apartado, incluye también una captura del cuadro de diálogo.
   - Ten en cuenta que el programa no termina hasta que cierres el cuadro de diálogo. Si no lo ves, puede haber quedado detrás de la ventana de IntelliJ.
4. **Gestión de excepciones:**
   - Implementa un método muestraErrorSQL para mostrar detalles del error en caso de que ocurra una SQLException. Este método debe mostrar:
     - El mensaje de error (getMessage()).
     - El estado SQL (getSQLState()).
     - El código de error del proveedor (getErrorCode()).
   - Captura cualquier otra excepción y muestra el mensaje correspondiente en la consola.

5. **Prueba del programa:**
   - Ejecuta el programa y comprueba que en la consola se muestra "Filas modificadas: 1" (o tantas como clientes tuvieras con el CP a NULL).
   - Comprueba en phpMyAdmin que ese cliente tiene ahora el CP "00000".
   - Ejecútalo una segunda vez y comprueba que se muestra "Filas modificadas: 0", porque ya no queda ningún CP a NULL.
   - Incluye una captura de cada ejecución y otra de la tabla en phpMyAdmin.

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Presenta la tarea, el código es correcto, pero no ha demostrado su funcionamiento mediante las capturas de pantalla
7	Presenta la tarea, el código es correcto, demuestra su funcionamiento mediante las capturas de pantalla, pero no describe los principales métodos incluidos en el código.
9	Se completa la tarea correctamente en todos sus apartados
10	Se completa la tarea correctamente y, además, se implementa la salida gráfica del apartado 3 u otra funcionalidad extra que mejore el programa.

Entrega:

1. **Incluye aquí el código fuente**:

```
package org.example;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import javax.swing.JOptionPane;

/**
 * @author Juan Gabriel Galarza Claros
 */

public class ModificarClientesResultSet {

    private static final String URL = "jdbc:mysql://localhost:3306/ut2";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        Connection connection = null;
        try {
            connection = DriverManager.getConnection(URL, USER, PASSWORD);

            // Deshabilitar auto-commit
            connection.setAutoCommit(false);

            // Configurar desplazamiento bidireccional y actualización
            try (Statement stmt = connection.createStatement(
                    ResultSet.TYPE_SCROLL_SENSITIVE,
                    ResultSet.CONCUR_UPDATABLE)) {
```

```
            ResultSet consulta = stmt.executeQuery("SELECT * FROM clientes");
            int nFil = 0;

            // Recorrer el ResultSet
            while (consulta.next()) {
                // Comprobar si el CP es nulo
                if (consulta.getString("CP") == null) {
                    consulta.updateString("CP", "00000"); // Actualizar valor
                    consulta.updateRow();                 // Guardar base de datos
                    nFil++;
                }
            }

            // Confirmar la transacción para guardar todos los cambios
            connection.commit();

            // Mostrar el resultado en consola
            String resultado = "Filas modificadas: " + nFil;
            System.out.println(resultado);

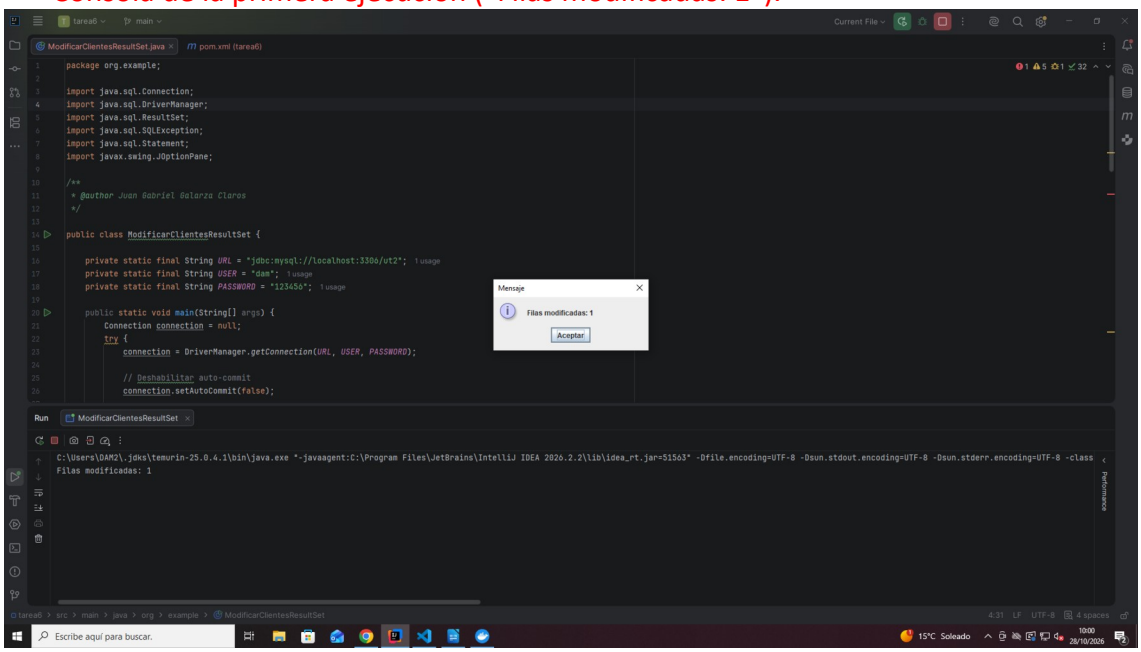
            JOptionPane.showMessageDialog(null, resultado);

        } catch (SQLException e) {
            // Deshacer cambios en caso de error
            connection.rollback();
            muestraErrorSQL(e);
        }
    } catch (SQLException e) {
        muestraErrorSQL(e);
    } catch (Exception e) {
        System.err.println("ERROR: " + e.getMessage());
    } finally {
        // Cerrar la conexión
        if (connection != null) {
            try {
                connection.close();
            } catch (SQLException ex) {
                ex.printStackTrace();
            }
        }
    }
  }

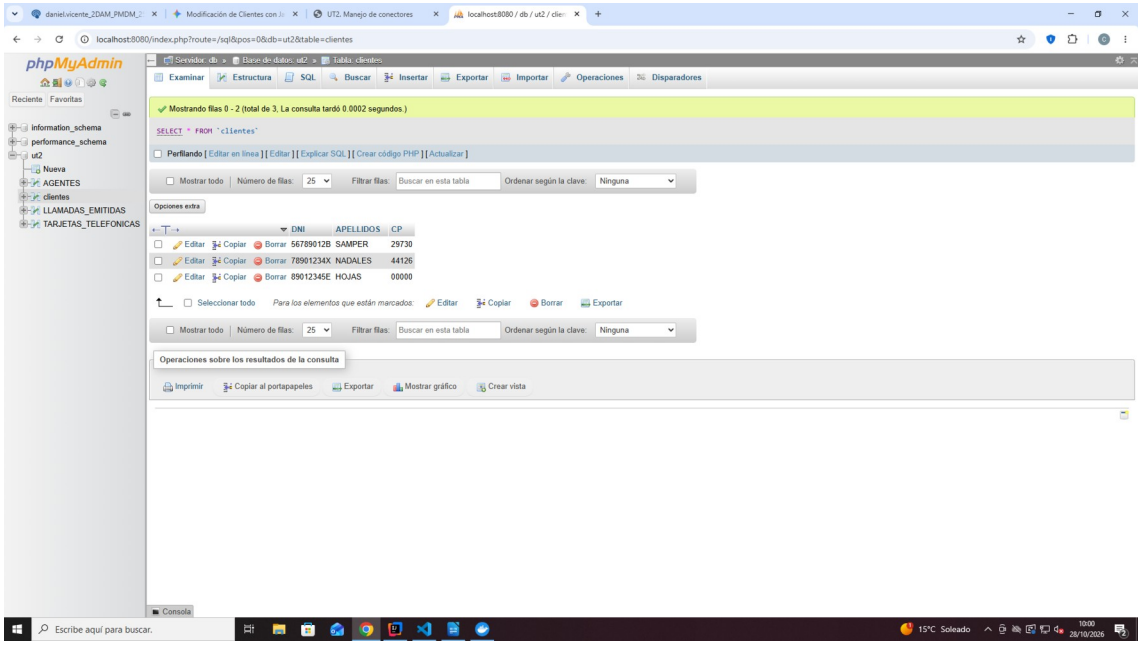
  // Metodo para mostrar detalles del error SQL
  private static void muestraErrorSQL(SQLException e) {
      System.err.println("Mensaje de error: " + e.getMessage());
      System.err.println("Estado SQL: " + e.getSQLState());
      System.err.println("Código de error: " + e.getErrorCode());
  }
}
```

2. **<u>Incluye aquí las capturas de pantalla que demuestren el correcto funcionamiento del programa</u>**:

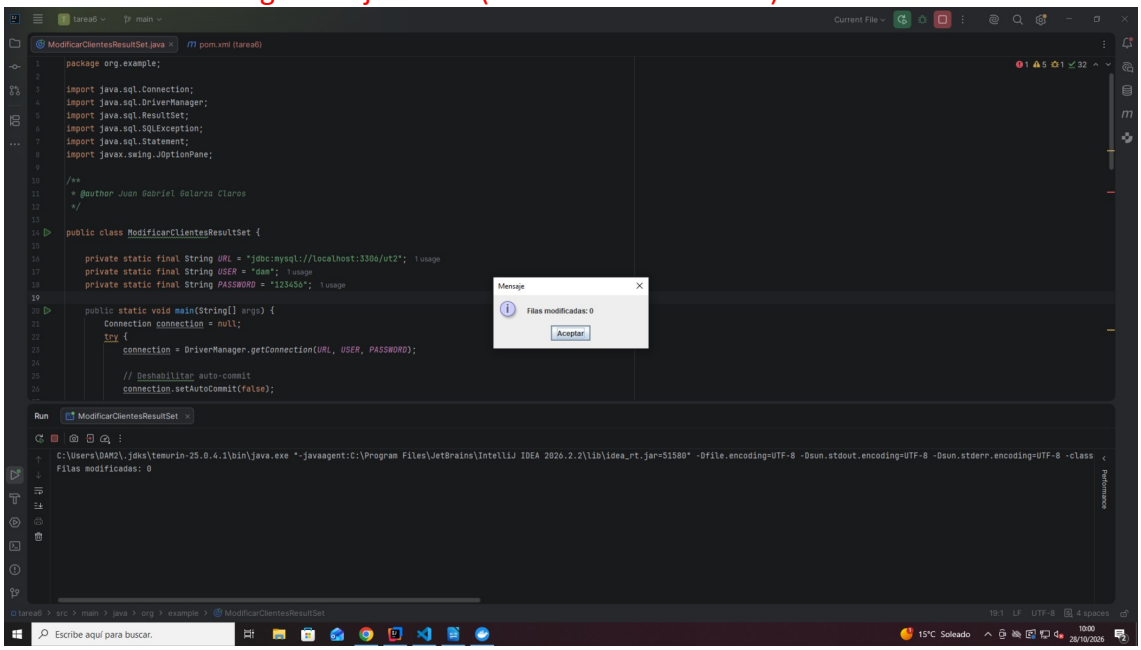
- Consola de la primera ejecución (“Filas modificadas: 1”).



- Tabla clientes en phpMyAdmin después de la primera ejecución, con el CP “00000”.



- Consola de la segunda ejecución (“Filas modificadas: 0”).



- Cuadro de diálogo, si has realizado el apartado 3.

3. **Explica con tus palabras qué hace cada uno de estos elementos y por qué es necesario en el programa: setAutoCommit(false), TYPE_SCROLL_SENSITIVE y CONCUR_UPDATABLE, updateString(), updateRow(), commit() y la variable nFil.**

setAutoCommit(false): desactiva el autoguardado en la base de datos.
TYPE_SCROLL_SENSITIVE: permite que el cursor de navegación se mueva hacia delante o atrás.
CONCUR_UPDATABLE: convierte el resulset en actualizable directamente sin sentencias update.
updateString(): actualiza el valor de una columna en la que esté el cursor.
updateRow(): acualiza la fila de la base de datos
commit(): guarda todo lo realizado anteriormente en la base de datos.
nFil: un variable que utilizamos como contador para saber cuantas filas hemos modificado.

DESCRIPCIÓN DE MÉTODOS, CLASES y VARIABLES

Clases :

ModificarClientesResultSet: es la clase principal que agrupa todo nuestro código para realizar la actividad.

Connection: la conexión entre nuestro programa y la base de datos MySQL.

DriverManager: es la herramienta que utiliza nuestras credenciales para establecer esa conexión inicial.

Statement: es el objeto que nos permite enviar las sentencias SQL a la base de datos.

ResultSet: es el objeto que almacena la tabla con los resultados que nos devuelve la consulta.

SQLException: es la clase encargada de capturar y manejar los errores específicos que ocurren al interactuar con bases de datos.

JOptionPane: es una clase gráfica de Java que usamos para dibujar la ventana emergente final.

Métodos:

main: es el punto de entrada automático donde empieza a ejecutarse todo el código.

getConnection: intenta conectar al servidor usando la dirección, el usuario y la contraseña.

setAutoCommit: decide si los cambios se guardan al instante o si los agrupamos para confirmarlos todos juntos al final.

createStatement: prepara el entorno de las sentencias y lo configuramos para permitir el modo actualizable.

executeQuery: lanza nuestra consulta de texto a la base de datos y nos devuelve los registros.

getString: extrae el texto de una columna específica para comprobar si está vacío.

updateString: cambia en la memoria el valor de texto de esa columna por el nuevo código postal.

updateRow: aplica ese cambio de la memoria y lo guarda en esa fila concreta de la base de datos física.

commit: confirma la transacción completa y hace permanentes todos los cambios.

rollback: deshace todas las modificaciones retenidas si ocurre algún fallo, dejando la base de datos como estaba.

showMessageDialog: crea y muestra físicamente la ventana emergente en la pantalla.

muestraErrorSQL: es un método propio que creamos para imprimir ordenadamente los detalles técnicos de un fallo.

Variables :

URL, USER y PASSWORD: almacenan los datos fijos que nos ha dado el enunciado para poder entrar al servidor.

connection: guarda la sesión activa de la base de datos para no perder la conexión durante el proceso.

stmt: guarda la orden SQL configurada que estamos preparando para enviar.

rs: guarda temporalmente el conjunto de clientes que nos devuelve la base de datos para poder recorrerlos.

nFil: es un número entero que usamos como contador para sumar un punto cada vez que arreglamos el código postal de un cliente.

mensaje: es una variable de texto donde juntamos la frase final con el número exacto del contador para mostrarlo por pantalla.